# The Concurrent Mindset

CSE 493L: Concurrency, Parallelism, and Rust

# Agenda for today

- 30 minutes of
  - About this course
  - About me
  - Defining concurrency and parallelism
- 20 minutes of Intro to Coroutines

# Welcome to Concurrency!

# About this course

- First offering
- Developed by me with Prof. Tom Anderson
  - With help from my wonderful TAs, Marcin Anforowicz and Oleg Ianchenko
  - As well as help from Yiqing Wang
- We will discuss concurrency, parallelism, C, Rust, computer architecture, web apps…

# About this course

- Weeks 1-3: Concurrency and Parallelism in C
- Weeks 4-5: Rust Intro
- Weeks 6-10: Concurrency and Parallelism & Rust

# About this course

- Weekly homeworks
  - Released Monday, due following Wednesday
- Some quizzes
- Final

5-10 hours / week of homework in programming assignments

There are people on the waiting list

# About Me

- 3rd year Ph.D.
- Advised by Tom Anderson & Baris Kasikci
- Research interests: operating systems, computer architecture

# About Me

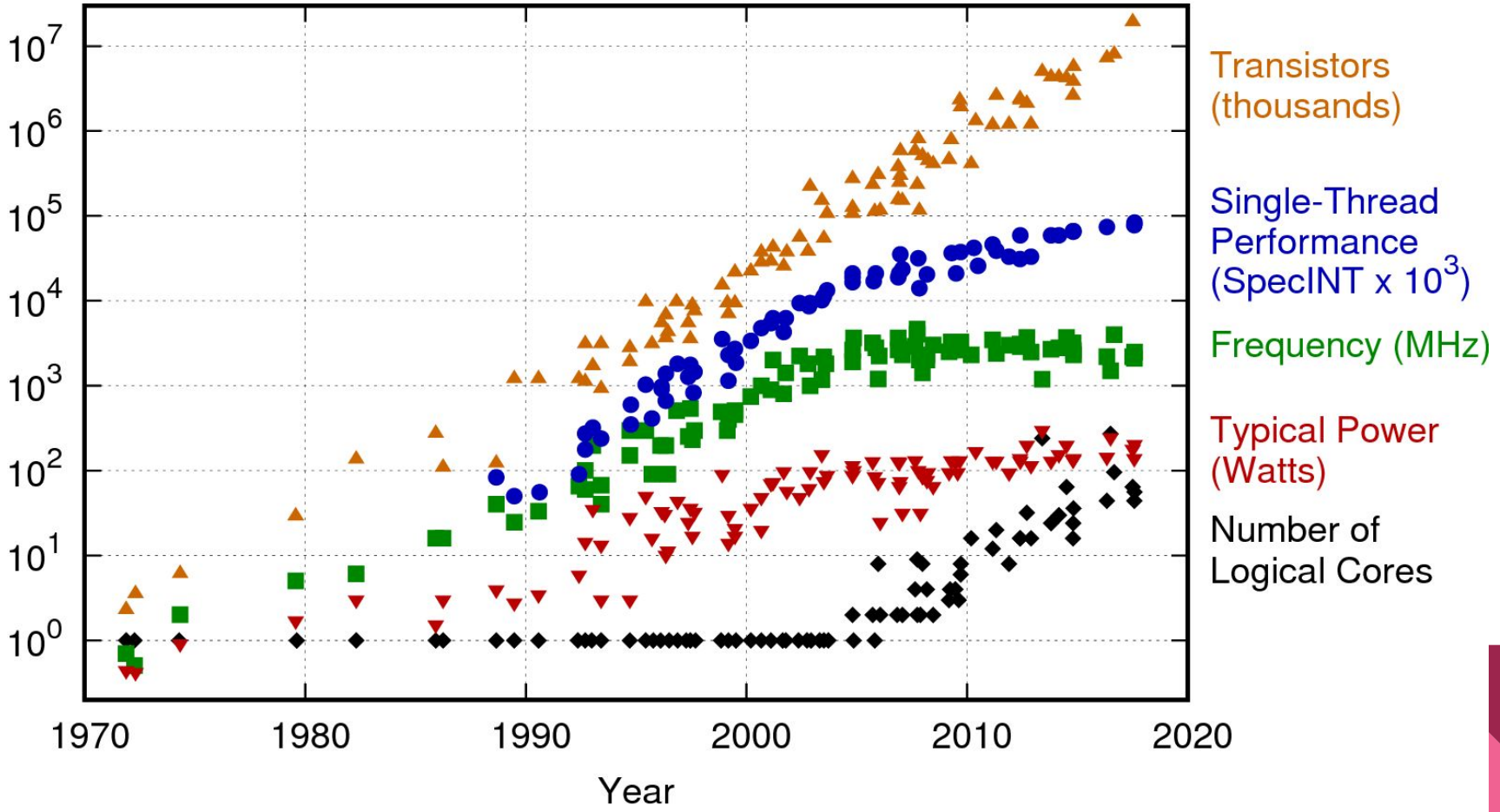
# Marcin & Oleg!

# Why Concurrency?

So that we can understand parallelism!

# Why Parallelism?

# 42 Years of Microprocessor Trend Data




Original data up to the year 2010 collected and plotted by M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, and C. Batten
New plot and data collected for 2010-2017 by K. Rupp

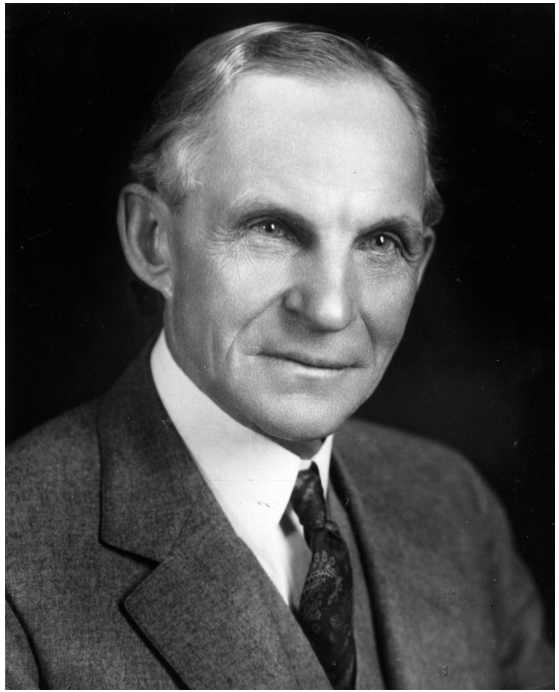
# Why Parallelism?

- Cores aren’t getting much faster (a few percent per year)
- Instead, we’re getting more of them!

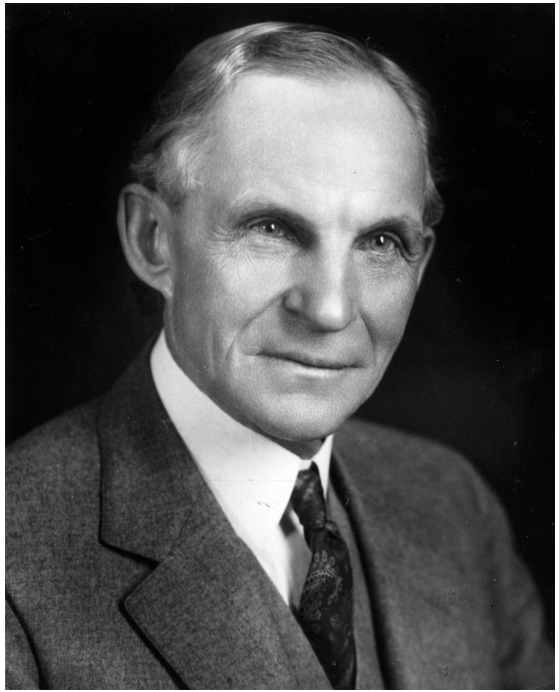
# How do we study parallelism?

Let’s look at a real-world example.

# Who’s this?



# Who’s this?



Smithsonian Magazine

In 1913, Henry Ford Introduced the Assembly Line: His Workers Hated It

Visit ›

Smithsonian Magazine
Smithsonian Magazine
Smithsonian Magazine
Smithsonian Magazine
In 1913, Henry Ford Introduced the Assembly Line: His Workers Hated It
In 1913, Henry Ford Introduced the Assembly Line: His Workers Hated It
Visit

# What did Henry Ford do?

He found a more efficient way to utilize his workers

He found a more efficient *execution* of the Model T's *dependencies* via specialization

Specialization is one technique to parallelize

This is one technique we will be exploring in this class, applied to code

# General Parallelism Techniques

How do you take advantage of more workers?

- Scale up
- Specialize

# Scale up

- Twice the number of workers = twice the number of Model Ts produced
  - how? just open another factory!
  - when does this fall short?

# Specialization

- 1 worker has to know how to do everything
- 2 workers only need to know half of the process!

# Specialization

Let’s go through an example that will illustrate:

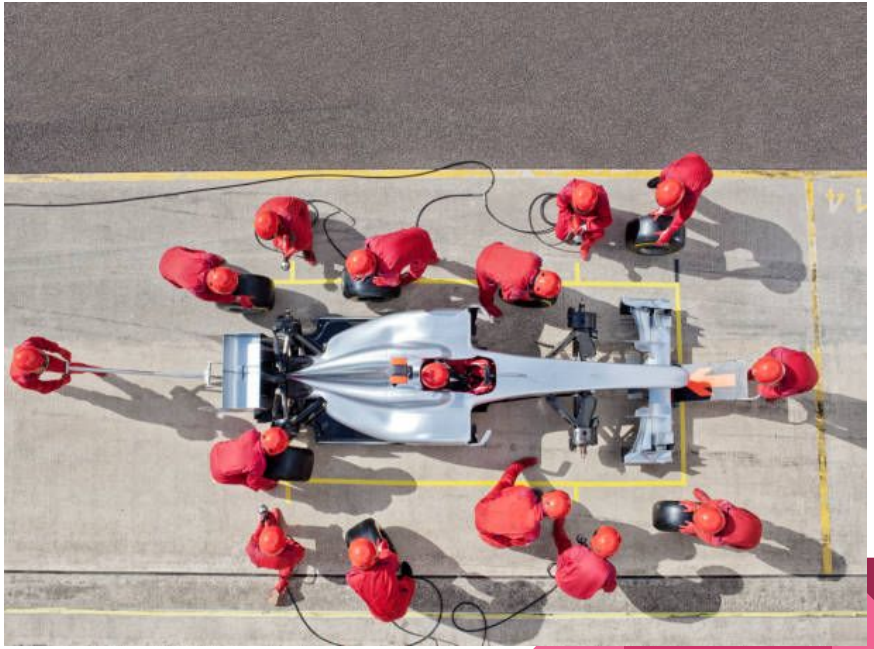
- Why is designing parallel algorithms hard?
- How do we formalize concurrency?
- Why is concurrency hard?

# Specialization

# Specialization

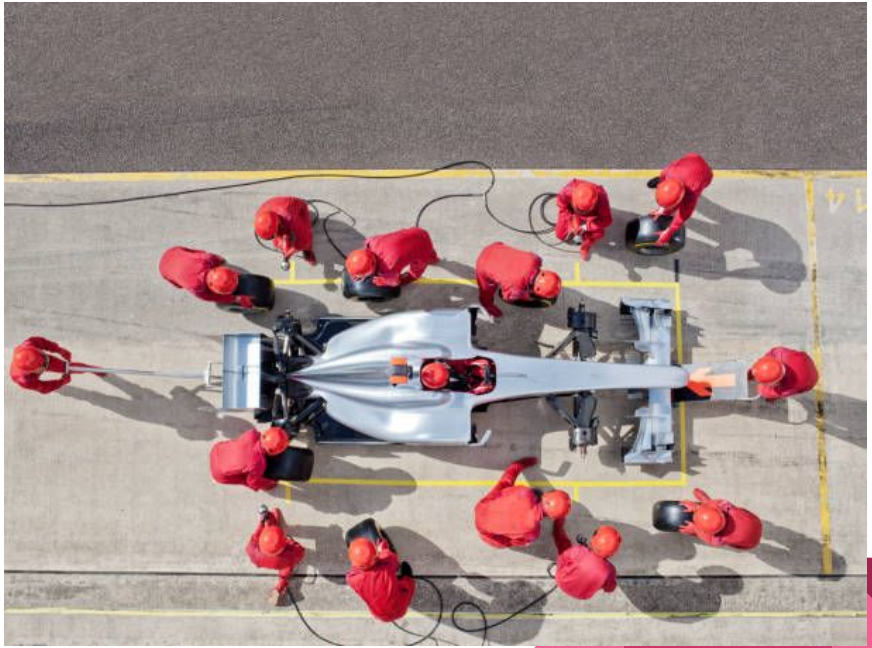
# You and a partner are going to create a Pit Stop Procedure

- Tasks:
  - Front jack up
  - Rear jack up
  - For each wheel:
    - Loosen wheel nut
    - Take old tire off
    - Put new tire on
    - Tighten wheel nut
  - Front jack down
  - Rear jack down
  - Tell driver to go (green light)



# What is the ideal protocol…

- With 1 person?
- With 2 people?
- With 6 people?



# Dependency Chain

- Tasks:
  - Front jack up
  - Rear jack up
  - For each tire:
    - Loosen wheel nut
    - Take old tire off
    - Put new tire on
    - Tighten wheel nut
  - Front jack down
  - Rear jack down
  - Tell driver to go (green light)

Can be executed concurrently

Each tire can be executed concurrently

Can be executed concurrently

# Dependency Chain

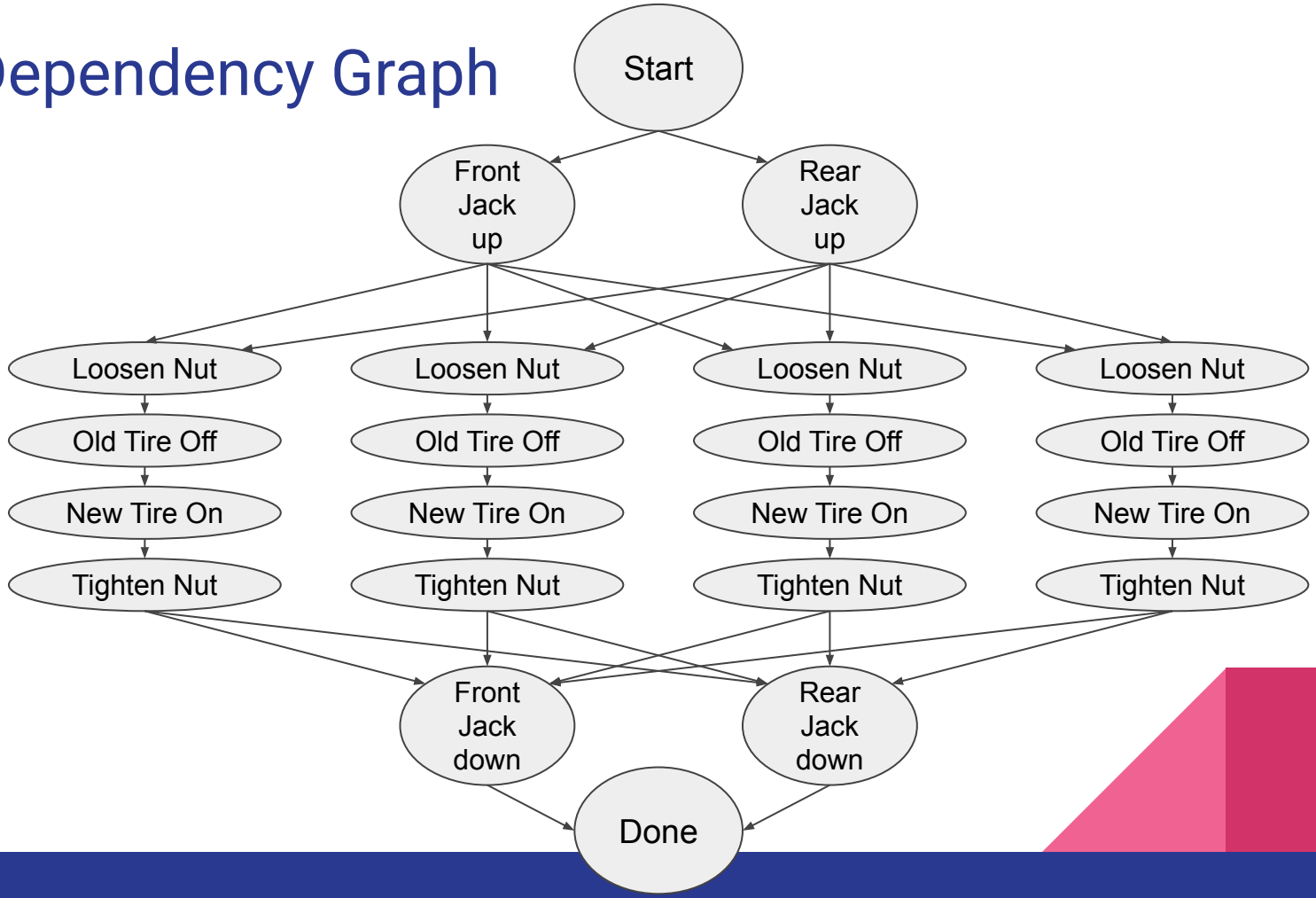
- Tasks:
  - Front jack up
  - Rear jack up
  - For each tire:
    - Loosen wheel nut
    - Take old tire off
    - Put new tire on
    - Tighten wheel nut
  - Front jack down
  - Rear jack down
  - Tell driver to go (green light)

Can be executed concurrently

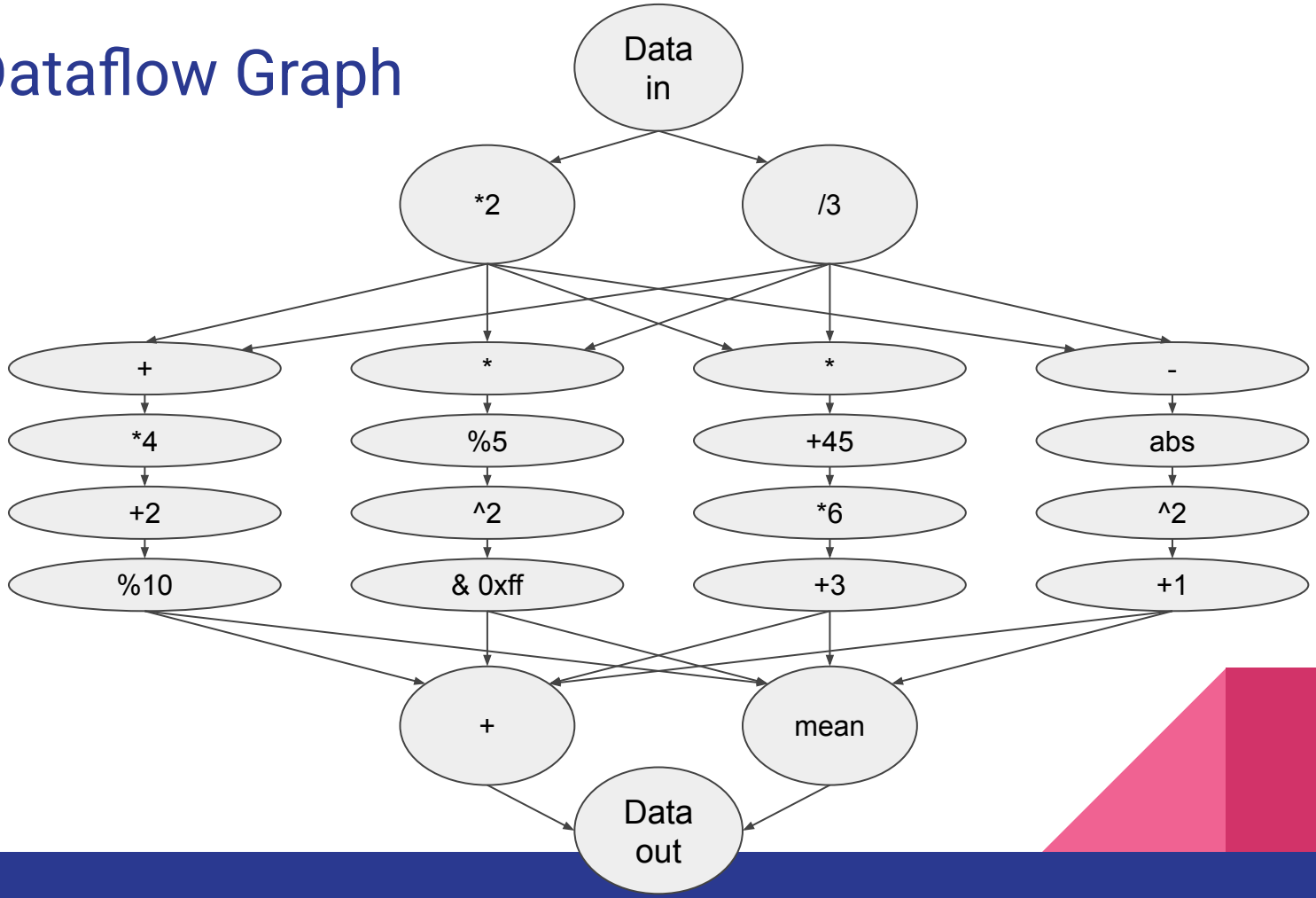
Each tire can be executed concurrently

Can be executed concurrently

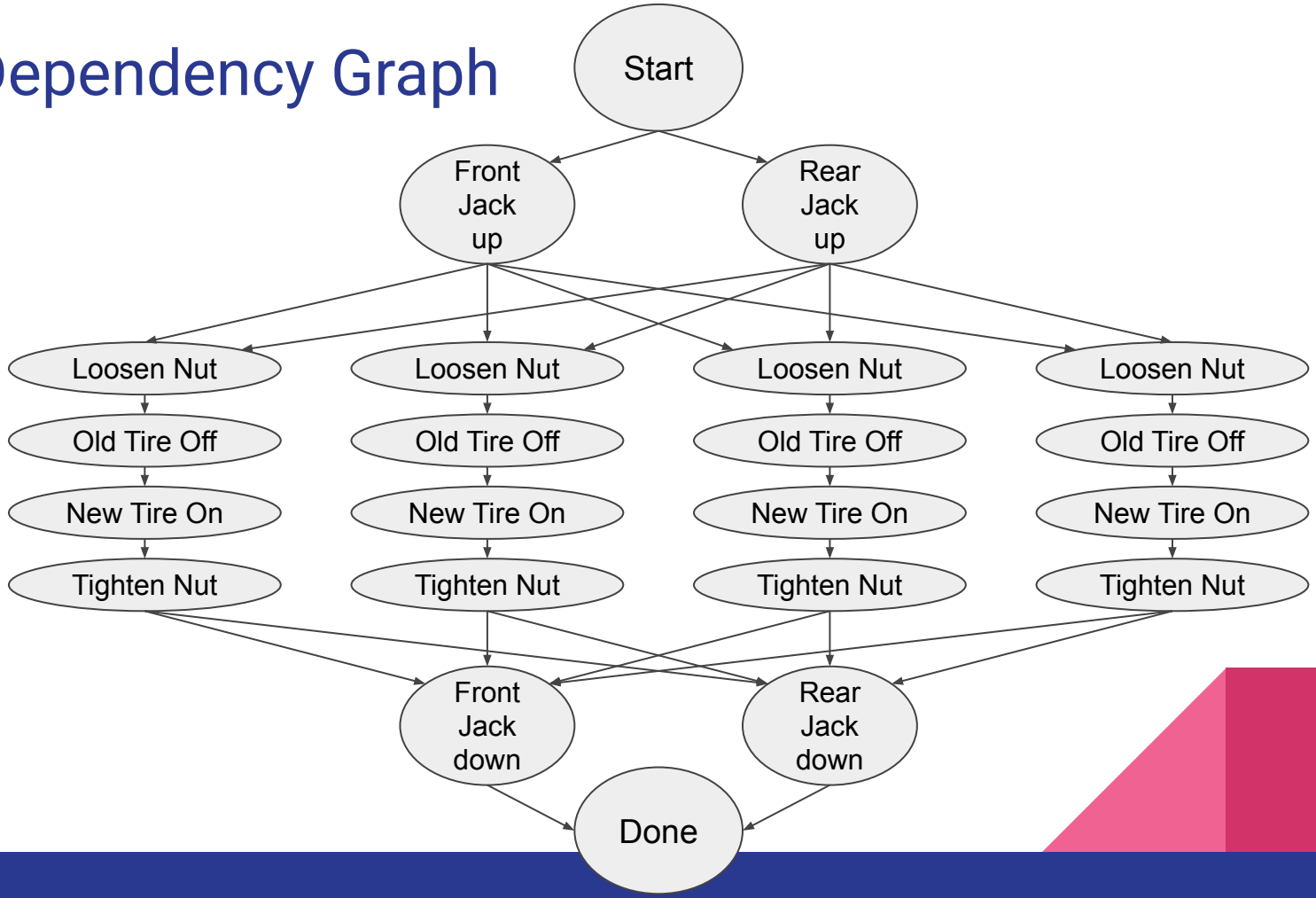
# Dependency Graph



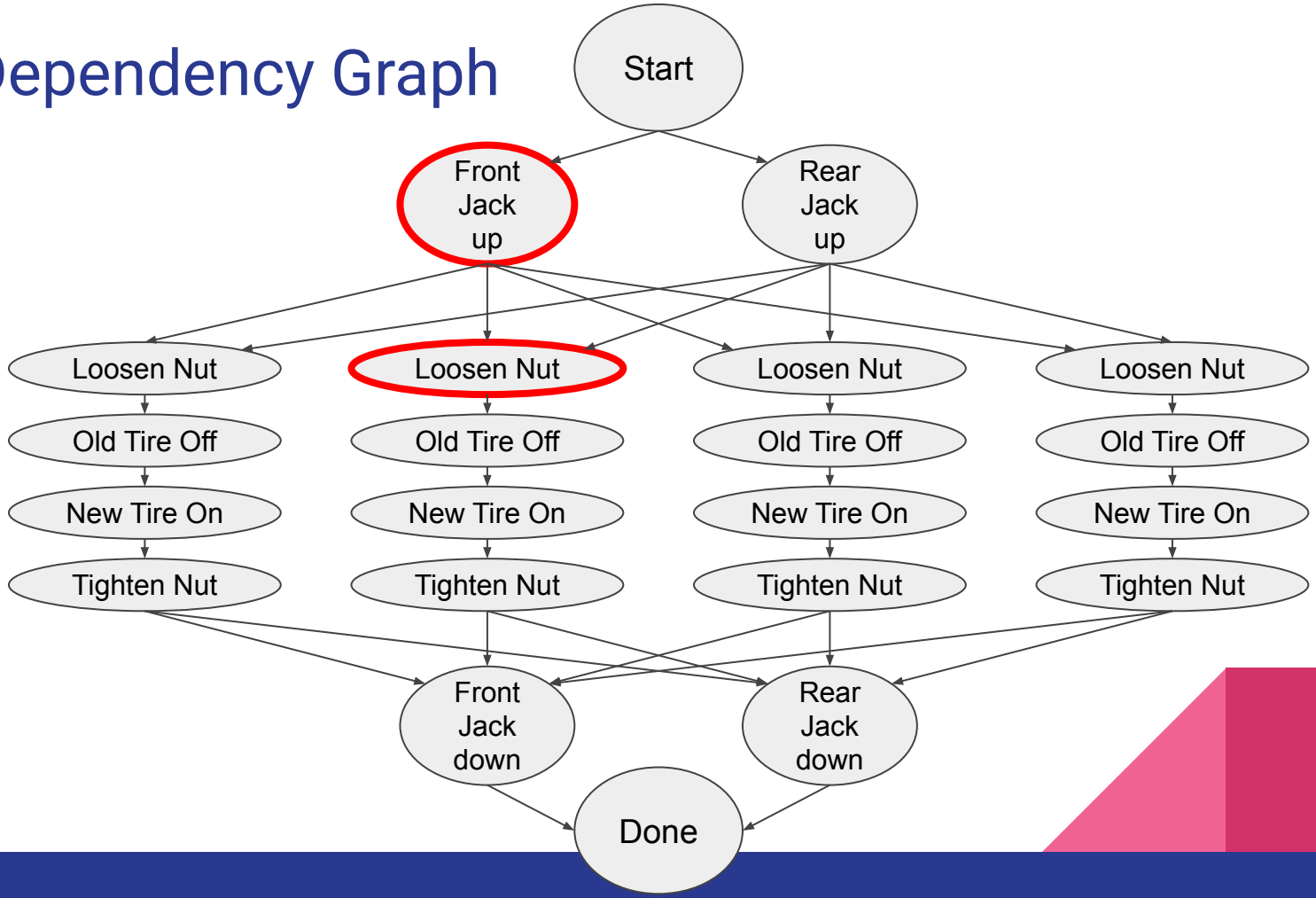
# Dataflow Graph



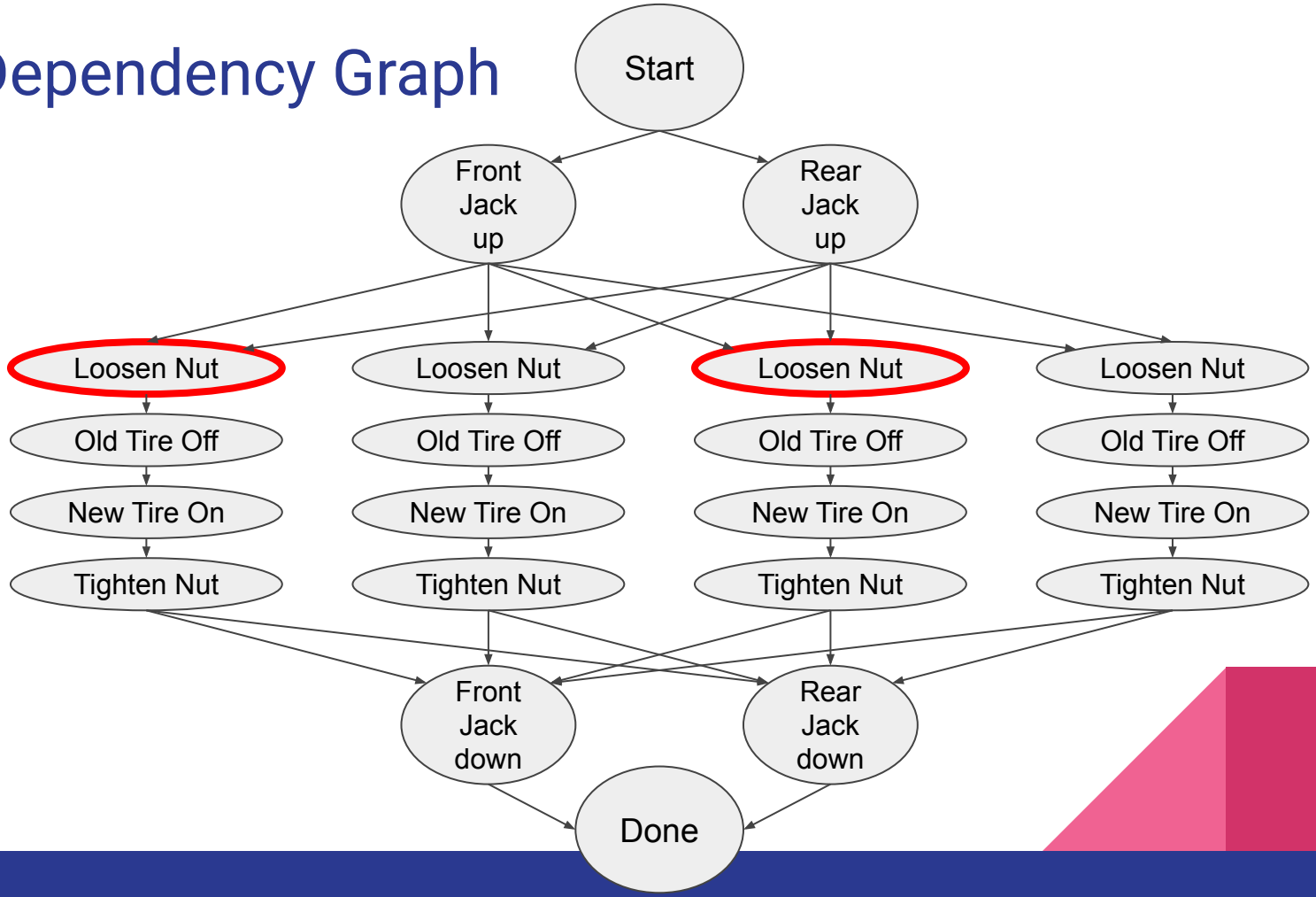
# Dependency Graph



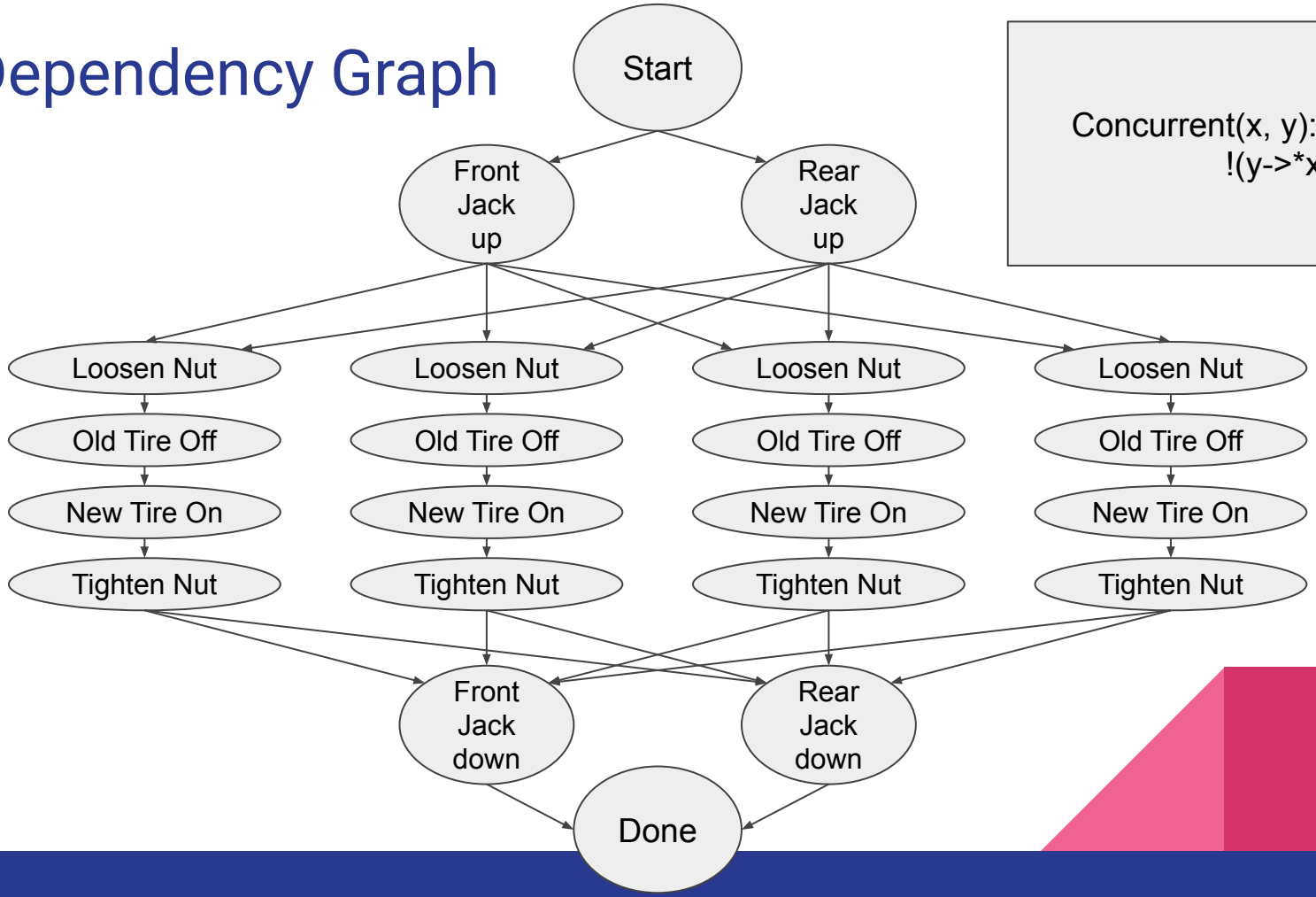
# Dependency Graph



# Dependency Graph

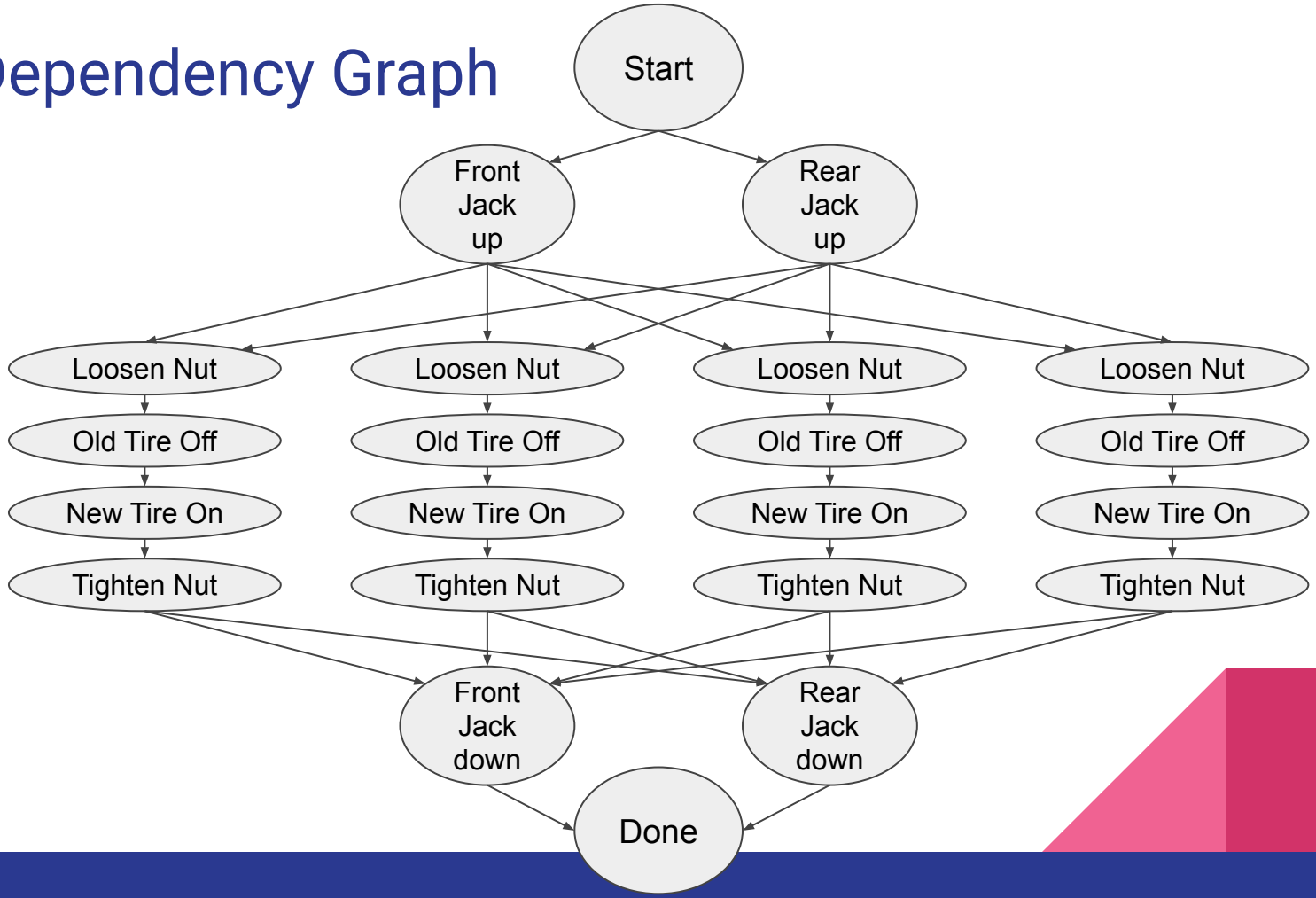


# Dependency Graph

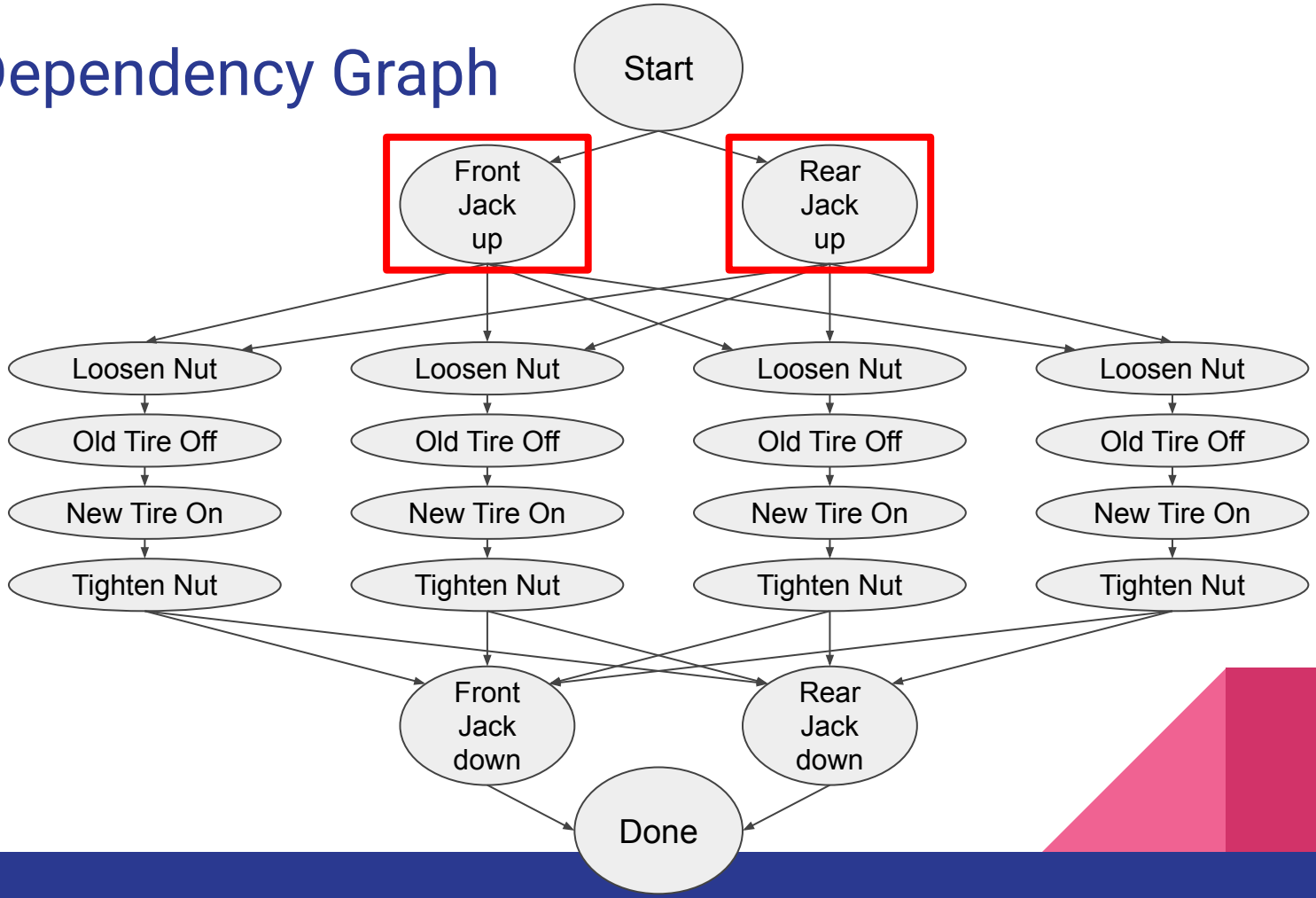



Concurrent(x, y): !(x->*y) && !(y->*x)

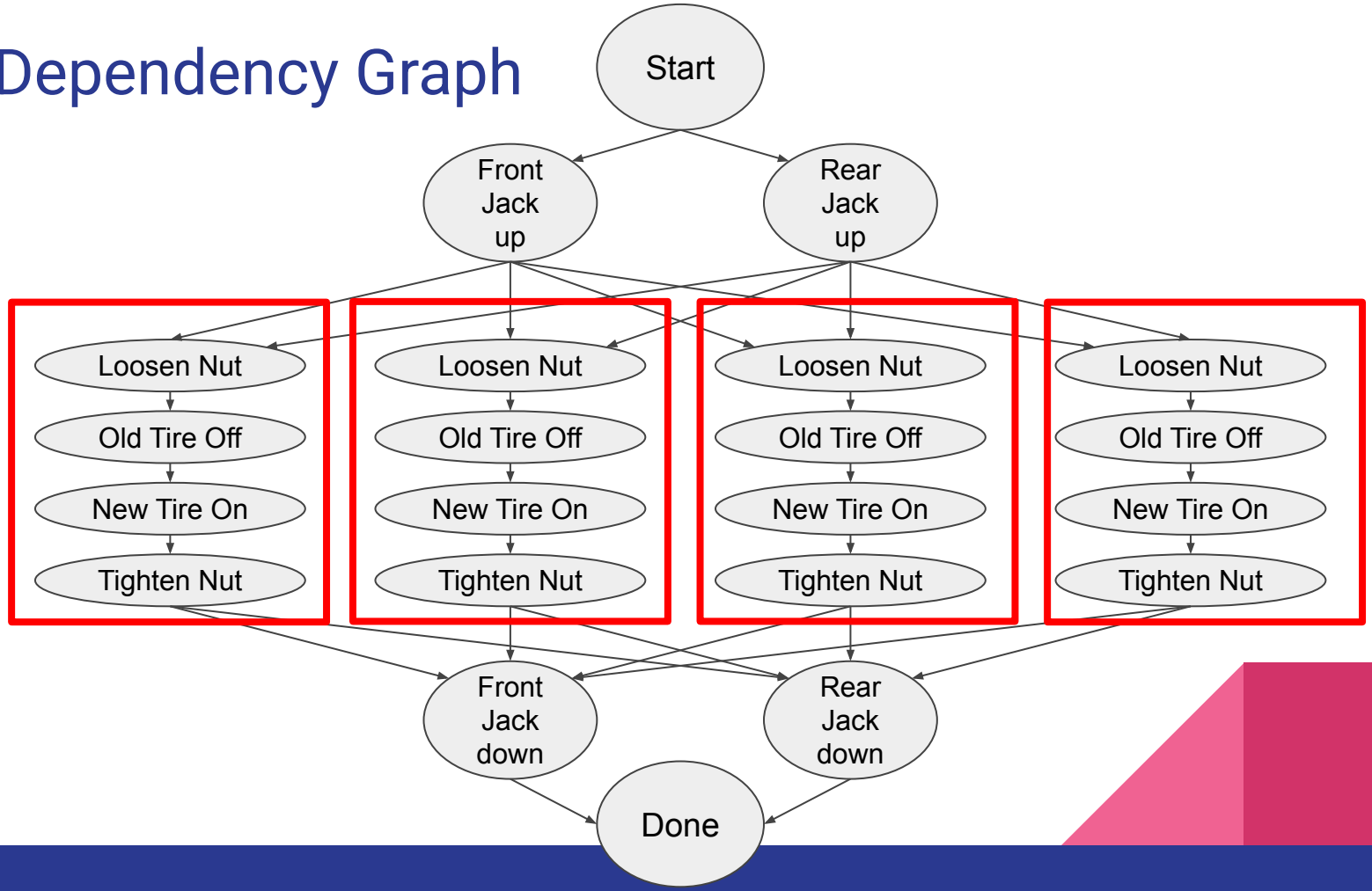
# Dependency Graph



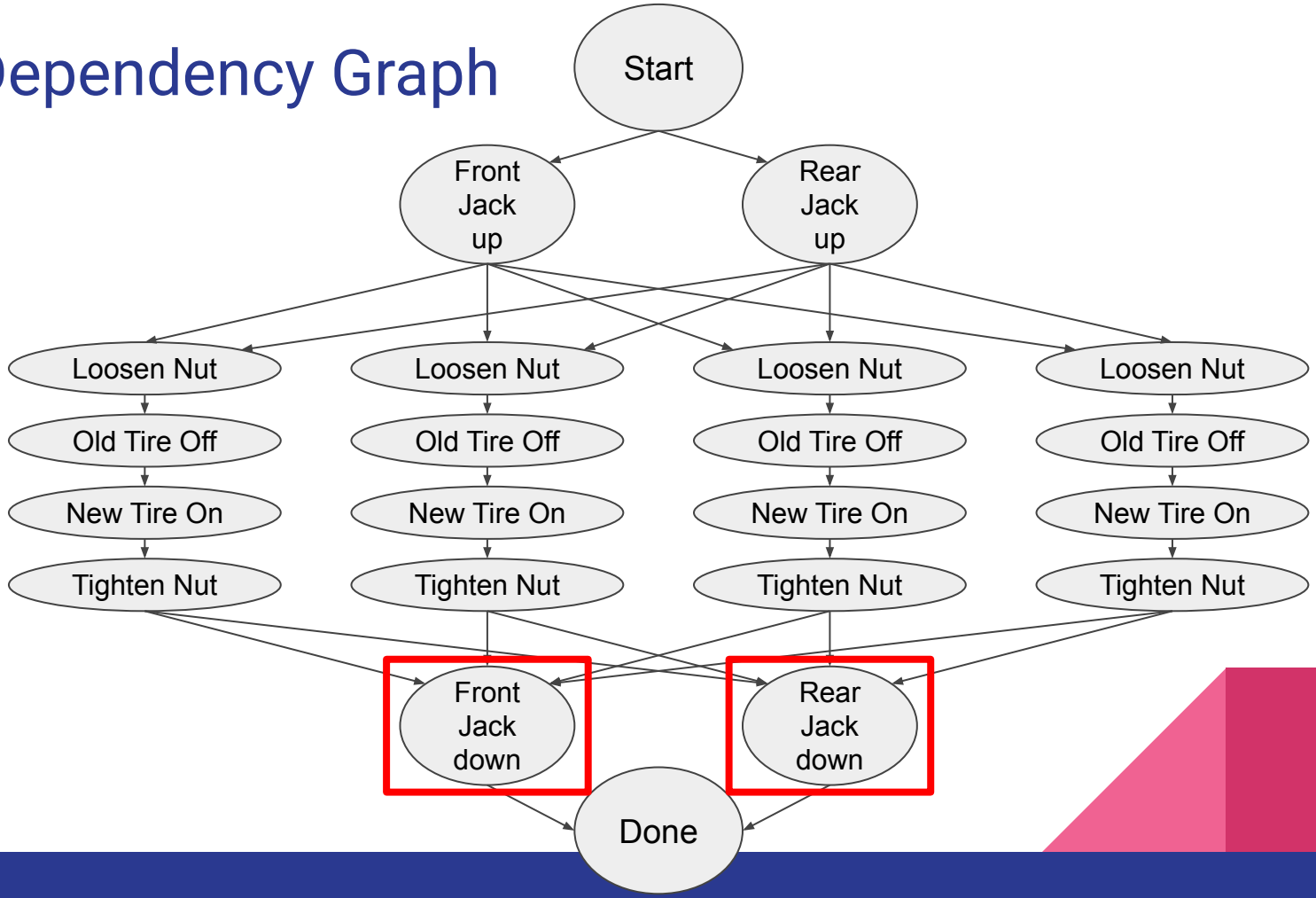
# Dependency Graph



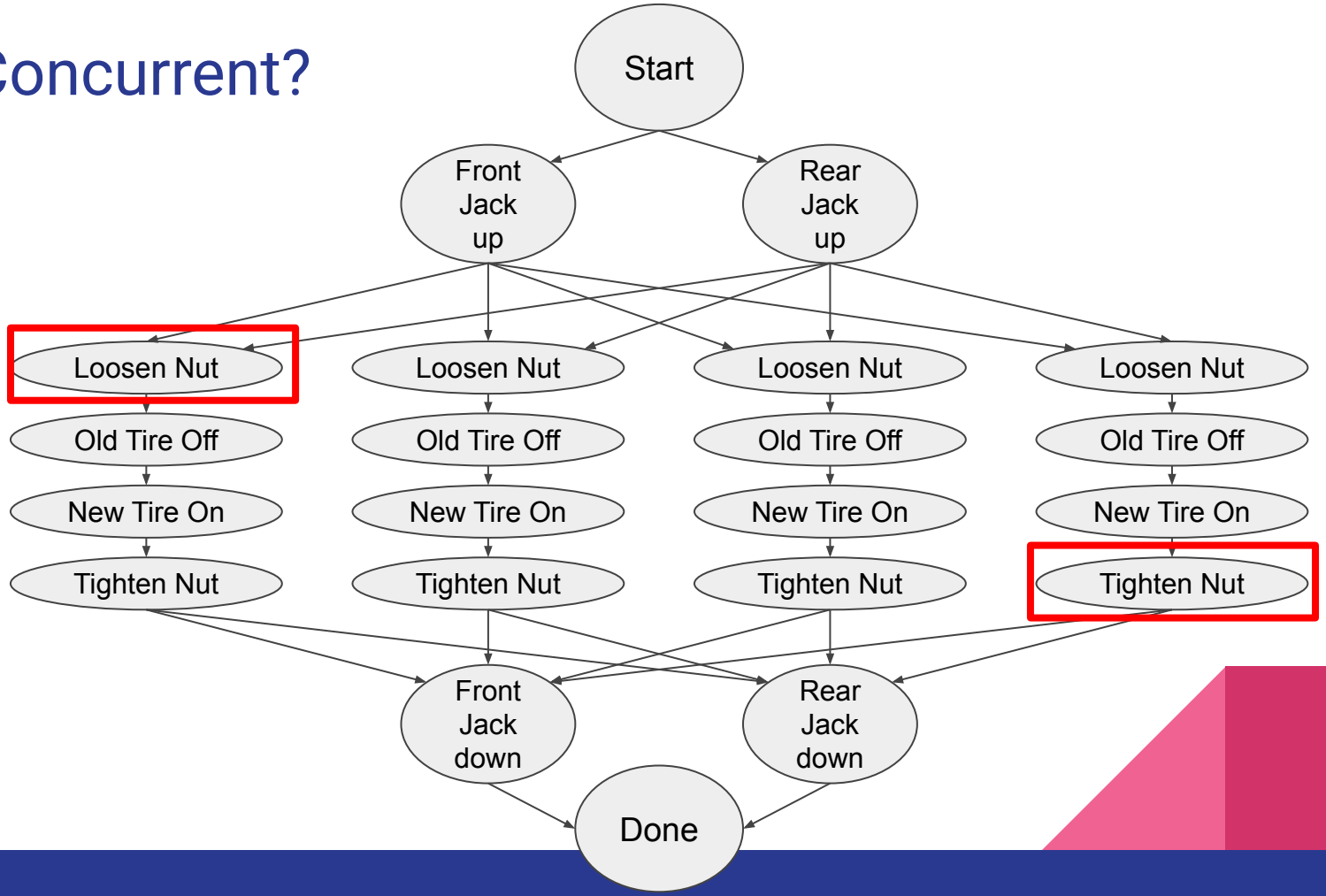
# Dependency Graph



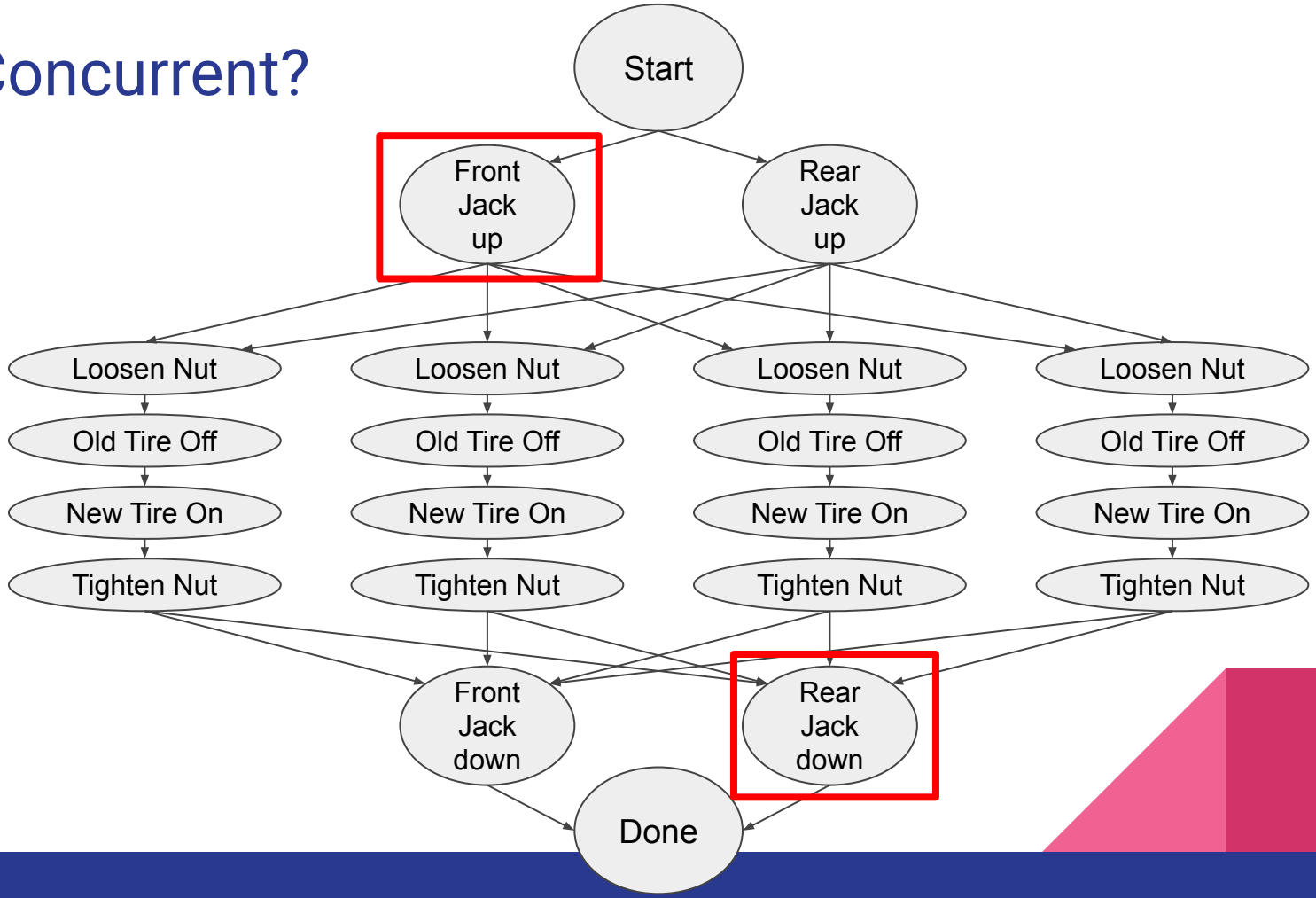
# Dependency Graph



# Concurrent?

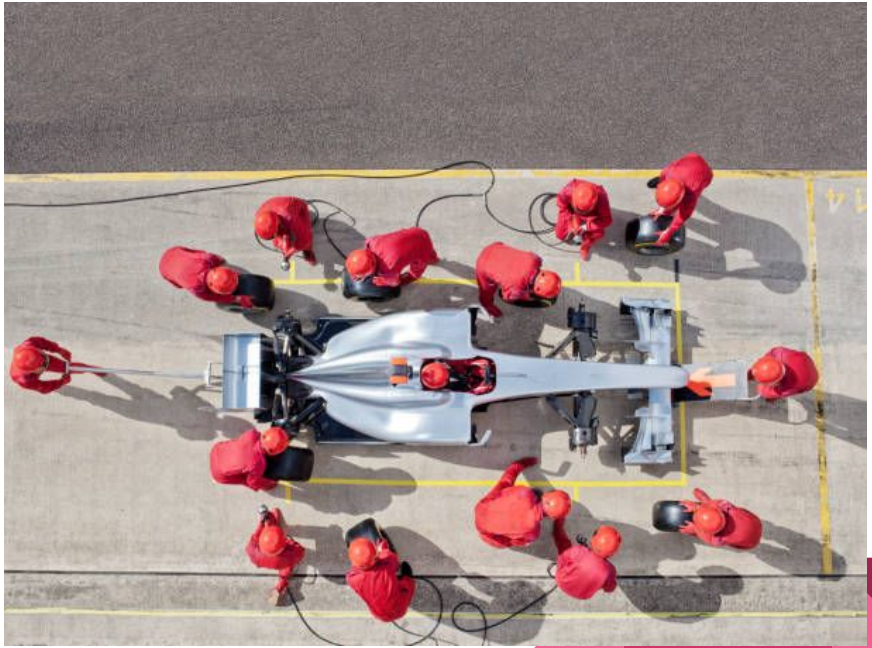

Start
Front Jack up
Rear Jack up
Loosen Nut
Loosen Nut
Loosen Nut
Loosen Nut
Old Tire Off
Old Tire Off
Old Tire Off
Old Tire Off
New Tire On
New Tire On
New Tire On
New Tire On
Tighten Nut
Tighten Nut
Tighten Nut
Tighten Nut
Front Jack down
Rear Jack down
Done

# Concurrent?


Start
Front Jack up
Rear Jack up
Loosen Nut
Loosen Nut
Loosen Nut
Loosen Nut
Old Tire Off
Old Tire Off
Old Tire Off
Old Tire Off
New Tire On
New Tire On
New Tire On
New Tire On
Tighten Nut
Tighten Nut
Tighten Nut
Tighten Nut
Front Jack down
Rear Jack down
Done

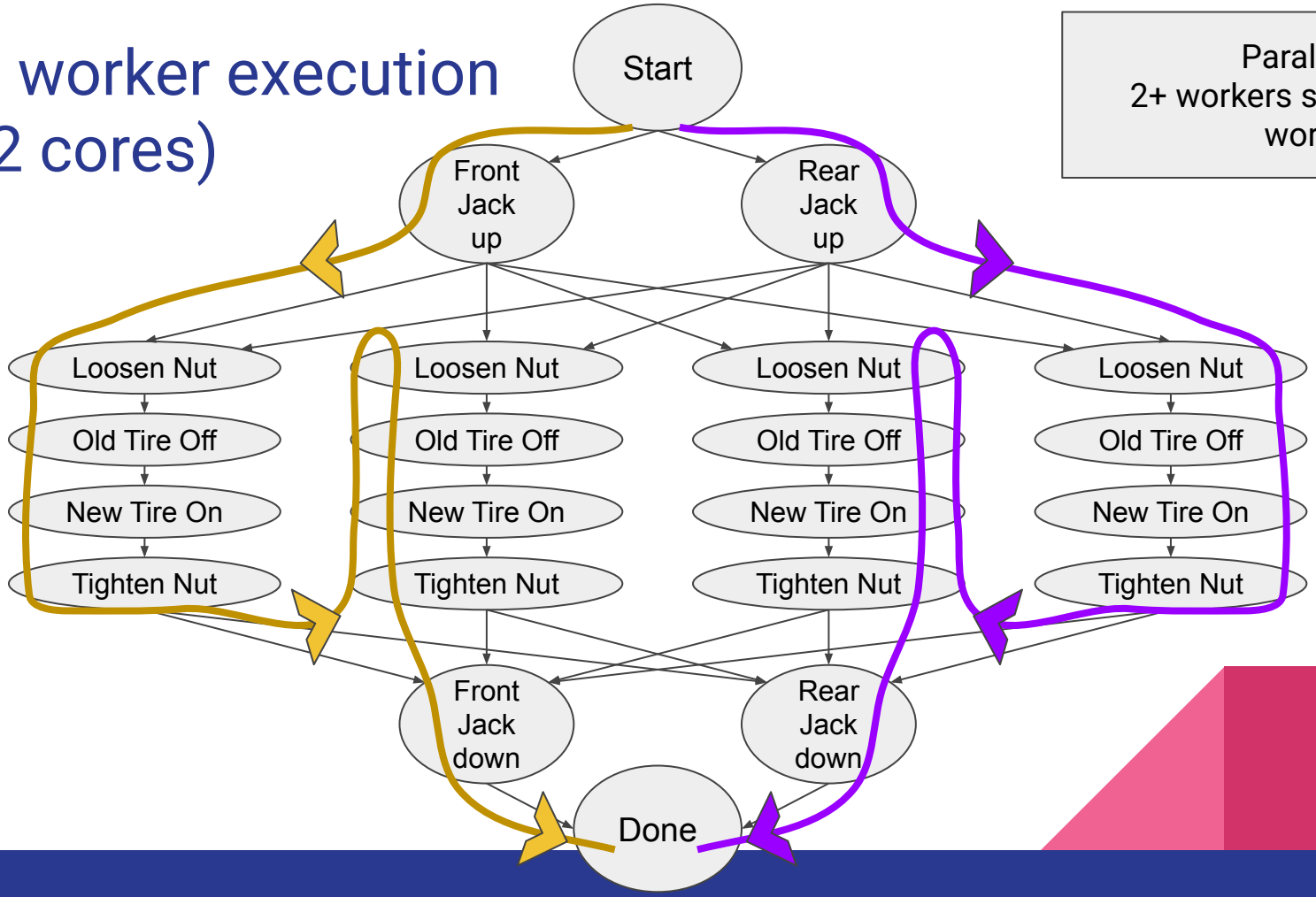
# How do we navigate this dependency graph?

- With 1 person?
- With 2 people?
- With 6 people?

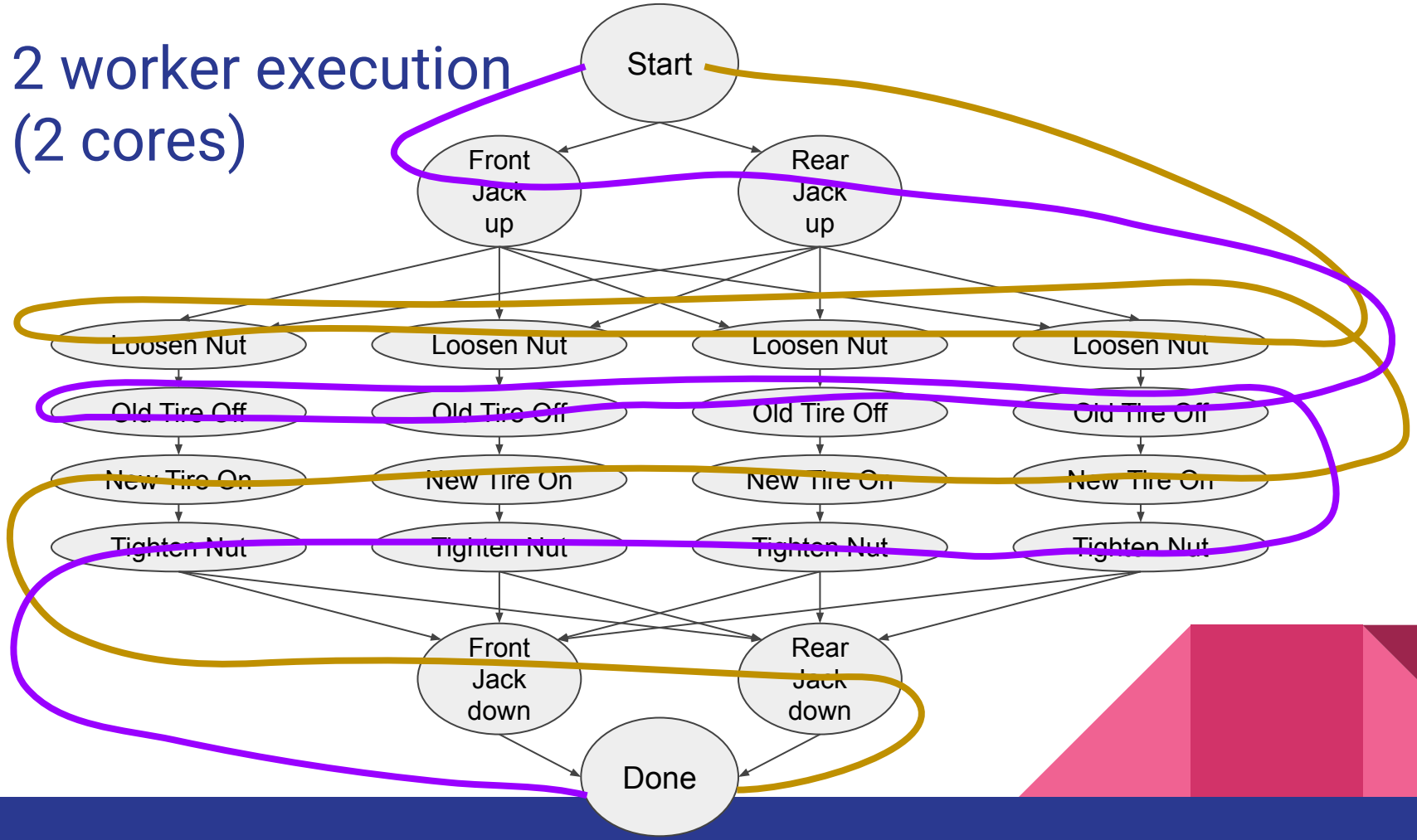


# 2 worker execution (2 cores)

Parallelism:
2+ workers simultaneously working!



# 2 worker execution (2 cores)



# Big dependency graphs are really hard to optimize well

- because there are O(|V|!) different ways to order the vertices
- Which means, more generally, that parallelizing code effectively (picking a good ordering) is really hard
  - Every new core is a new worker! What do you do with it?

# Writing parallel code in the first place is hard!

- False cacheline sharing
- Data races
- Race conditions
- Deadlocks
- Memory management bugs

# …which is why you should take this class!

Hopefully I can teach you:

- how to navigate the massive state space of choosing a parallel execution via useful abstractions
- how to write good concurrent & parallel code

# Other reasons you should take this class

- Parallelism is becoming ever-more important as high core counts become ubiquitous
- Concurrent reasoning is critical in distributed or asynchronous systems
- Rust is the future of systems programming, and we will do most of our assignments in Rust
  - Rust's compiler prevents many concurrency related bugs
  - Once you learn Rust, you never want to touch C/C++ again!

# Any Questions?

Moving on to hw1 material: coroutines!